[FIELD GUIDE]

Doing Less, On Purpose.

How the right Rails skills made my AI coding bill *12× smaller* — and made "done" actually mean done.

> *Ten real Rails tasks. Two ways of using AI. $144.72 on the meter. Every diff read by hand.*

Abraham Kuri — CTO & co-founder, Coba (YC S23) · kurenn.dev

00 READ THIS FIRST

The improvement isn't that skills do more. It's that a good one knows when to do less.

Most Rails AI tools answer every ticket the same way: spin up a team. A model specialist, a controller specialist, a security specialist, a test specialist — each its own process, each re-introducing itself, each on the meter. For a typo.

I spent **$144.72** measuring what that costs. Ten real Rails tasks, run twice: once through that spin-up-a-team approach, once through a leaner one that sizes the work up first and dispatches only what the ticket actually needs. Same models, same tasks. I read every diff by hand.

The lean approach didn't win by being smarter. It won by being disciplined. That's the whole guide, and everything after this is the receipts.

RECEIPT — The full run — every task, cost, time, token count, and diff — is public: github.com/kurenn/roundhouse/blob/main/BENCHMARK.md

01 THE SETUP

Two ways of using the same models on the same work.

THE SWARM

An architect agent decomposes every task and spawns specialists as separate Claude Code processes, coordinating over MCP. This is claude-on-rails v0.4 — at the time, the best-known Rails agent system on GitHub.

TRIAGE-FIRST

An orchestrator classifies each task into one of three tiers — trivial / single-domain / cross-cutting — before it does anything, then dispatches only what's needed. A typo bypasses the team and the orchestrator just edits the file. Single-domain work gets exactly one specialist. Cross-cutting work gets the full treatment: a plan, tests first, parallel implementers, then security and database gates only if the change touches them. Crucially, its specialists are subagents inside one session, not separate processes.

Same task text. Same models — Opus orchestrator, Sonnet specialists. Same prompt-refinement step on both sides. Test bed: a Rails 8.0.3 app with PostgreSQL, Tailwind, RSpec and Devise. Ten tasks, from "fix a typo in a flash message" to "add an admin API with auth and rate limiting." Every run logged cost, wall-clock, tokens, and the resulting diff.

RECEIPT — Both systems are open source. Swarm: github.com/kurenn/claude-on-rails · Triage-first: github.com/kurenn/roundhouse

02 THE RESULT

7× to 34× cheaper. 2× to 7× faster. Same working code, both sides.

TASK	TIER	SWARM	ROUNDHOUSE	ROUNDHOUSE WINS BY
Fix flash typo	trivial	$9.10 3m9s	**$0.36** 30s	**25.6× cheaper** 6.3× faster
Add missing translation	trivial	$11.90 2m44s	**$0.35** 23s	**33.6× cheaper** 7.1× faster
Unique-slug validation + index	single-domain	$13.19 5m26s	**$0.70** 2m42s	**18.9× cheaper** 2.0× faster
Post.recent scope	single-domain	$10.01 4m39s	**$0.61** 2m3s	**16.5× cheaper** 2.3× faster
Extract service object	single-domain	$19.78 8m15s	**$1.68** 4m25s	**11.8× cheaper** 1.9× faster
Write missing User specs	tests-only	$6.71 2m15s	**$0.48** 1m25s	**14.1× cheaper** 1.6× faster
Add Comment resource	cross-cutting	$21.93 14m4s	**$1.52** 6m30s	**14.4× cheaper** 2.2× faster
Publish notification job	cross-cutting	$12.95 6m54s	**$1.83** 4m22s	**7.1× cheaper** 1.6× faster
Admin API + auth + rate limit	cross-cutting	$18.17 16m20s	**$2.65** 8m55s	**6.9× cheaper** 1.8× faster
Sync→async report refactor*	cross-cutting	$4.55 2m34s	$6.25 33m38s	*swarm shipped no code; roundhouse shipped 16 files / 34 specs*
Totals (all 10)		**$128.29** 66 min	**$16.43** 65 min	**7.8× cheaper overall**

* Not an error — an asterisk. The one task where the lean system cost more, because the swarm quit after planning and shipped nothing. Section 05 tells that story.

Across the nine tasks both systems actually completed: **12.2× cheaper, 2.1× faster** — same working code, every diff read by hand. The tenth task is the interesting one; hold that thought.

RECEIPT — This table, with token counts and cache behavior per run, is BENCHMARK.md.

03 WHY

It's not a prompt problem. It's a process-boundary problem.

The multiplier is the least interesting number here. This is the part I care about.

I broke the swarm's bill down by who spent it — the architect versus its specialists. On average, **69% of the cost was the architect talking to its own team.** On a task that was nothing but writing specs, it hit 85.5%.

69%	85.5%
average share of the bill spent coordinating, not coding	peak — on a task that was nothing but writing specs

Here's the mechanism, and it's almost boringly mechanical. When the swarm spawns a specialist, it spawns a separate process with its own context — one that can't share the parent's cache. So every specialist boots cold and re-pays roughly **50,000 tokens** just to re-introduce itself: the system prompt, the tool schemas, all of it. Every time the architect turns to coordinate, it re-pays its overhead too. Subagents running inside one session share that cache. That single architectural fact is most of the 12×.

The models weren't working harder. The architecture was making them re-introduce themselves to each other, over and over, and charging me for the introductions.

RECEIPT — Same task, side by side: output tokens differ under 2×, cache traffic differs 2.5×. The full token profile is in BENCHMARK.md.

04 THE COUNTERINTUITIVE PART

The savings are biggest on the easy tickets.

I assumed the gap would be widest on the hard tasks. It's the exact opposite.

~29×	~14×	~9×
TRIVIAL TASKS	SINGLE-DOMAIN	CROSS-CUTTING

The advantage shrinks as the work gets harder — because on genuinely cross-cutting work, some of that coordination is real work that has to happen. On a typo, none of it is.

Which produced the single most absurd line in the data: the swarm spent $9.10 and three minutes fixing a one-character typo in a flash message. It correctly identified a code change and did exactly what it was built to do — spawned a controller specialist, a security specialist, a test specialist. They booted. They coordinated. They reported back. For one character. The triage-first system looked at it, called it trivial, and edited the file: $0.36, 30 seconds.

> *The expensive failure mode in AI coding isn't the agent that can't solve the hard problem. It's the agent that takes an easy problem seriously.*

Most of your tickets are typos. A copy change, a missing index, a scope. If your system convenes a committee for those, you're paying committee prices for clerical work — on the majority of your volume.

RECEIPT — The $9.10-vs-$0.36 typo is row one of the table. It is not a hypothetical.

05 THE FAILURE THAT TAUGHT ME THE MOST

A green test suite that had never run the work.

The hardest task: an async report refactor. The swarm's architect took it and planned — well, a thorough 250-line spec. Then it ran out of budget and stopped. It never delegated to a single specialist. Not one line of code was written. $4.55, zero output.

And my harness reported:

```
tests_pass: true
```

Which was true. The pre-existing specs passed — because nothing had changed. The suite was green because the work had not been done. I nearly logged it as a success. The only reason I caught it is that I was diffing every run by hand, and the diff was empty.

LESSON ONE

More planning is not free. We've absorbed "make the agent think harder" as if thinking were costless. It isn't — planning spends the same budget as doing, and an agent that spends its whole allowance on a magnificent plan has shipped nothing.

LESSON TWO

Your success signal probably can't tell "it worked" from "it did nothing." DORA's latest data says 31% of PRs now merge with no human review at all. Go and check what your definition of done actually proves.

> *Passing tests measure the absence of regression, not the presence of work.*

For the record: my own worst run was this same task — $6.25 and 33 minutes, the one task where I was more expensive than the swarm. It shipped, slowly and badly. It's on the table. I'm not hiding it.

RECEIPT — The empty diff and the tests_pass: true line are both in BENCHMARK.md, task T3.4.

06 WHAT A GOOD SKILL ACTUALLY DOES

Discipline, made mechanical.

Strip away the benchmark and here's what "using skills well" means in Rails — four rules, and none of them is "use a smarter model":

1 **Triage before dispatch.**
Decide the size of the work before you spend on it. Most tickets don't need a team, and the ones that do, you'll know.

2 **Tests first — where it counts.**
Cross-cutting work runs test-first. A typo doesn't. TDD is a tool, not a tax you pay on every change.

3 **Gates on demand.**
Security and database review fire only when the change touches their concern — not on every commit, where they're just more coordination.

4 **Receipts, always.**
A skill that returns "done" with a file path and a diff you can read beats one that narrates what it probably did. If you can't check it, you didn't automate it.

RECEIPT — These are the actual rules roundhouse runs on, written down in its README and DECISIONS.md.

07 TRY IT ON YOUR CODEBASE

It's open source. Point it at a real ticket.

roundhouse is the triage-first system from this benchmark — a Rails team delivered as Claude Code skills (models, controllers, services, jobs, views, Tailwind, tests, and a bug-fix flow), orchestrated by triage. Free, Apache-licensed.

```
$ claude plugin marketplace add kurenn/marketplace
$ claude plugin install roundhouse@kurenn
```

Give it one real ticket. Read the diff it produces. Then read the bill. That's the only benchmark that matters — the one on your own code.

RECEIPT — roundhouse: github.com/kurenn/roundhouse · the full study: BENCHMARK.md · the write-up: kurenn.dev

The cheapest 12× in Rails is doing less — on purpose.

Abraham Kuri — Mexico City · kurenn.dev
I publish the receipts, including when I'm wrong.